

Circuit Documentation

Summary

This circuit is designed to monitor environmental conditions using a combination of sensors and an ESP32-S3 microcontroller. It includes a water level sensor, a rain sensor, and a force sensor. The ESP32-S3 microcontroller reads data from these sensors and sends the information to a remote server using WiFi. The circuit is powered by the ESP32-S3's 3.3V output.

Component List

1. **Water Level Sensor**
 - **Description:** Measures the level of water.
 - **Pins:** SIG, VCC, GND
2. **YL-83 Rain Sensor**
 - **Description:** Detects the presence and intensity of rain.
 - **Pins:** POS, NEG
3. **Force Sensor**
 - **Description:** Measures the force or pressure applied to it.
 - **Pins:** pin 1, pin 2
4. **ESP32-S3 (beta)**
 - **Description:** A powerful microcontroller with WiFi capabilities.
 - **Pins:** 3V3, RST, 4, 5, 6, 7, 15, 16, 17, 18, 8, 3, 46, 9, 10, 11, 12, 13, 14, 5V, GND, 19, 20, 21, 47, 48, 45, 0, 35, 36, 37, 38, 39, 40, 41, 42, 2, 1, RX, TX
5. **Resistor (10k Ohms)**
 - **Description:** Used to limit current and divide voltage.
 - **Pins:** pin1, pin2
6. **Resistor (10k Ohms)**
 - **Description:** Used to limit current and divide voltage.
 - **Pins:** pin1, pin2

Wiring Details

Water Level Sensor

- **VCC:** Connected to ESP32-S3 pin 3V3
- **GND:** Connected to ESP32-S3 GND and both resistors' pin2
- **SIG:** Connected to ESP32-S3 pin 2

YL-83 Rain Sensor

- **POS:** Connected to ESP32-S3 pin 3V3
- **NEG:** Connected to ESP32-S3 pin 1 and one resistor's pin1

Force Sensor

- **pin 1:** Connected to ESP32-S3 pin 3V3
- **pin 2:** Connected to ESP32-S3 pin 3 and one resistor's pin1

ESP32-S3 (beta)

- **3V3:** Supplies power to the Water Level Sensor, Force Sensor, and YL-83 Rain Sensor
- **GND:** Common ground connection for the Water Level Sensor and resistors
- **Pin 2:** Reads signal from Water Level Sensor
- **Pin 1:** Reads signal from YL-83 Rain Sensor
- **Pin 3:** Reads signal from Force Sensor

Resistors

- **Resistor 1 (10k Ohms)**
 - **pin1:** Connected to ESP32-S3 pin 3 and Force Sensor pin 2
 - **pin2:** Connected to ESP32-S3 GND
- **Resistor 2 (10k Ohms)**
 - **pin1:** Connected to ESP32-S3 pin 1 and YL-83 Rain Sensor NEG
 - **pin2:** Connected to ESP32-S3 GND

Code Documentation

The code for the ESP32-S3 microcontroller is written in C++ and is designed to read sensor data, process it, and send it to a remote server using WiFi. Below is a breakdown of the code functionality:

- **Libraries:** The code includes the WiFi.h library to handle WiFi connections.
- **Pin Definitions:**
 - RAIN_PIN is defined as pin 1.
 - WATER_PIN is defined as pin 2.
 - FSR_PIN is defined as pin 3.
- **WiFi Credentials:** The SSID and password for the WiFi network are defined.
- **Persistent Values:** Initial values for water and force are set.
- **Setup Function:** Initializes serial communication and connects to WiFi.
- **Loop Function:**
 - Reads analog values from the sensors.
 - Maps raw sensor values to meaningful ranges.
 - Implements accumulation logic based on rain intensity.
 - Combines sensor readings with manual control values.
 - Constrains the water and force values within specified limits.
 - Prints the sensor data to the serial monitor.
 - Sends the data to ThingSpeak using an HTTP GET request.

The code is structured to run continuously, updating sensor readings and sending data every 15 seconds.

```
#include <WiFi.h>

#define RAIN_PIN 1
#define WATER_PIN 2
#define FSR_PIN 3

const char* ssid = "CirkitWifi";
const char* password = "";

String apiKey = "LGK5RFBKM9XS9TWWF";

// 🔥 Persistent values (important)
float water = 200;
float force = 250;

void setup() {
    Serial.begin(115200);

    WiFi.begin(ssid, password);
    Serial.print("Connecting to WiFi");

    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }

    Serial.println("\nWiFi Connected!");
}

void loop() {

    // 💎 Read sensors
    int rawRain = analogRead(RAIN_PIN);
    int rawWater = analogRead(WATER_PIN);
    int rawForce = analogRead(FSR_PIN);
```

```

// ♦ Map values
int rain = map(rawRain, 0, 4095, 0, 100);
int waterBase = map(rawWater, 0, 4095, 100, 400);
int forceBase = map(rawForce, 0, 4095, 150, 500);

// 🔥 Accumulation logic (KEY PART)
if (rain > 50) {
    water += rain * 0.1;    // gradual increase
    force += rain * 0.15;
}
else if (rain > 20) {
    water += rain * 0.05;
    force += rain * 0.08;
}
else {
    water -= 2;    // evaporation / release
    force -= 3;
}

// ♦ Combine with manual control (very important)
water = (water + waterBase) / 2;
force = (force + forceBase) / 2;

// ♦ Limit values
water = constrain(water, 100, 500);
force = constrain(force, 150, 600);

// ♦ Print
Serial.print("Rain: "); Serial.print(rain);
Serial.print(" | Water: "); Serial.print(water);
Serial.print(" | Pressure: "); Serial.println(force);

// ♦ Send to ThingSpeak
String url = "/update?api_key=" + apiKey +
    "&field1=" + String(rain) +
    "&field2=" + String((int)water) +
    "&field3=" + String((int)force);

```

```

WiFiClient client;

if (client.connect("api.thingspeak.com", 80)) {
  client.print(String("GET ") + url + " HTTP/1.1\r\n" +
    "Host: api.thingspeak.com\r\n" +
    "Connection: close\r\n\r\n");
}

delay(15000);
}

```

